

Updating the World Is Not Rebinding the Target: Temporal Referent Integrity in Tool-Using Agents

Anonymous submission

Abstract

Refreshing external state does not by itself authorize a tool-using agent to change the entity denoted by an earlier reference. We formalize this post-binding problem as *temporal referent integrity* (TRI), separating belief updates from referent transitions. On a frozen 160-task diagnostic, a Generic Structured Ledger reaches 64.4%/71.9% with Qwen3.5-122B/GLM-5.1, whereas pre-refresh Compile-then-act (CTA) reaches 95.0%/96.2%. Always-Lock and Always-Reevaluate each reach 60.0% and fail opposite modes. In a separately frozen 240-task replication, conditional on correct initial binding, Generic drifts on 43/72 and 38/80 opportunities; CTA and Lifecycle-Gated drift on none, and all 81 Generic drifts replay as wrong-entity SQLite writes. A typed Lifecycle reaches 98.1%/100.0% on the primary set, but its gate adds only 1.2–1.9 points over a matched free actor. Three blind annotators obtain Fleiss’ $\kappa = .708$ and 86% majority-gold agreement on 100 original/rewrite items. Public ToolSandbox, AppWorld, and τ^3 audits find almost no strict TRI opportunities, and external full-history studies observe zero conditional TRI. TRI is therefore a controlled, model- and controller-conditional mechanism diagnosis, not a prevalence or safety claim.

Introduction

Consider two requests issued before the same mailbox refresh:

- (1) “Choose the highest-priority unread email now. Refresh the mailbox, then reply to it.”
- (2) “Refresh the mailbox first. Then choose the highest-priority unread email and reply.”

Suppose email A is highest priority before refresh and email B afterward. The environment transition is identical, but the correct target differs. Request (1) establishes A as an identity commitment; request (2) deliberately leaves a description to be evaluated in the new state. Acting on B in (1) is not stale-state correction. It is a wrong-entity action.

This distinction is easy to lose in LLM agent controllers. A controller may preserve the entire transcript, append the latest observation, and ask a downstream model to act. All facts remain available, yet the model can silently apply the

old selector to the new state. Conversely, always retaining an old ID breaks genuinely dynamic references. Reliable tool use therefore needs *selective invariance*: world facts should update while some target denotations remain fixed, others are reevaluated, and invalid commitments follow an explicit fallback policy.

We call this property *temporal referent integrity* (TRI). TRI is not ordinary entity memory. Remembering an ID does not say whether that ID is a committed referent or merely the previous output of a dynamic selector. Nor does existence imply action validity: an email may remain present but cease to be replyable. Finally, the selector that established identity is not necessarily a persistent validity predicate. A branch chosen because it was default can stop being default while remaining the correct branch on which to run checks.

Figure 1 summarizes the problem and our controller. A Generic Structured Ledger records identity, facts, selector, and action preconditions, while leaving the actor to infer how those fields interact across refresh. Minimal Compile-then-act (CTA) first records binding time and identity. Its typed Lifecycle-Gated extension adds validity and fallback state; a deterministic gate enforces preserved identities while an actor resolves dynamic branches.

Our contributions are threefold:

- We formalize TRI as a post-binding authorization problem separating reference mode, bound identity, selector provenance, action-relative validity, and guarded fallback. Latest-state observations can be informationally insufficient; full history may contain the answer while still leaving the commitment computationally implicit and unenforced.
- We introduce frozen clustered diagnostics for language-template clusters, schema transfer under shared language templates, and guarded conditional commitments. We pre-specify cluster-resampled statistics and audit API failures, missing tasks, retries, and stage-wise errors.
- Across two models, independent human rewrites, SQLite mutations, and public trace audits, component evidence most strongly supports pre-refresh commitment compilation. Minimal CTA has the highest or tied scalar point estimate; typed lifecycle state exposes a richer authorization contract, while negative stress tests localize its compilation and compositional costs.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

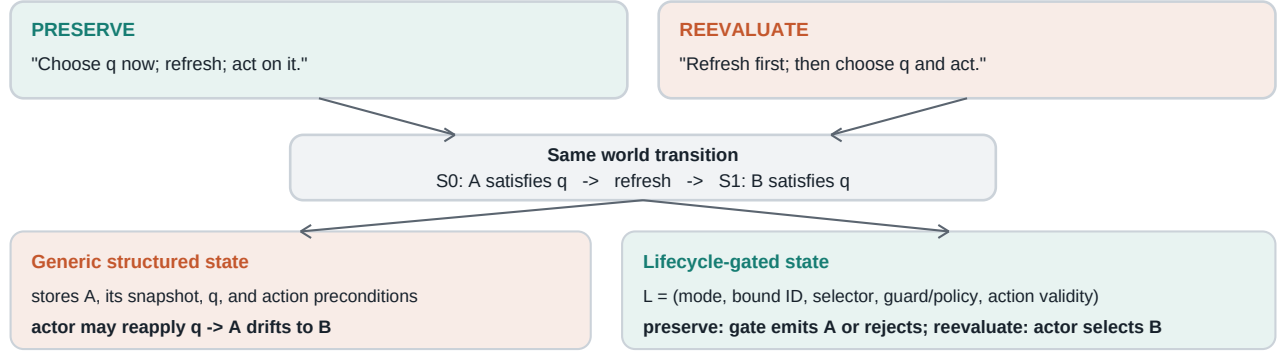


Figure 1: The same state transition requires different targets depending on discourse. Generic structured state preserves facts but leaves the interaction between identity and selector implicit. Lifecycle state makes the permitted denotation update executable.

Related Work

Reference, binding, and memory. Discourse theories distinguish referents from changing descriptions and salience (Kamp and Reyle 1993; Grosz, Joshi, and Weinstein 1995); coreference work recovers textual identity links (Lee et al. 2017; Joshi et al. 2020; Dasigi et al. 2019). Entity Binding Failures studies wrong initial tool arguments (Babu and Indukuri 2026); TRI instead conditions on correct $bind(r, S_0)$ and asks whether S_1 licenses a transition. Agent memory and entity tracking retain facts or traces (Tang et al. 2026; Packer et al. 2023; Summers et al. 2024; Yao et al. 2023), but an ID alone does not state whether it is committed or a stale selector result.

Binding Drift. The closest neighbor studies silent entity replacement in multi-step workflows and evaluates locking and re-verification (Indukuri 2026). TRI overlaps with its Preserve branch but also includes authorized Reevaluate cases: “choose now, refresh, act” and “refresh, then choose” require opposite targets under the same transition. We reproduce its deterministic offline harness, but its oracle re-verifier reads a gold target; its Entity Lock corresponds to our Always-Lock extreme. TRI therefore studies authorization of transition, not persistence alone.

Tool agents and executable constraints. Tool-use benchmarks cover APIs, web environments, and stateful interaction (Schick et al. 2023; Li et al. 2023; Qin et al. 2024; Patil et al. 2023; Liu et al. 2024; Zhou et al. 2024; Lu et al. 2025; Trivedi et al. 2024; Yao et al. 2024; Barres et al. 2025; Sierra Research 2026). Their gold does not label post-binding Preserve/Reevaluate opportunities. LedgerAgent and Bounded Autonomy add typed state or action contracts (Uddin et al. 2026; Sohail and Haider 2026); action validity still cannot determine which valid entity discourse authorized. Structured generation can enforce syntax (Beurer-Kellner, Fischer, and Vechev 2023; Willard and Louf 2023; Khattab et al. 2024), while our clustered minimal pairs follow diagnostic and contrast-set methodology (Ribeiro et al. 2020; Gardner et al. 2020).

Problem Formulation: Temporal Referent Integrity

Let u be an instruction, r its action-target reference, S_0 the state before refresh, S_1 the refreshed state, and a the requested action. We factor the executable contract into a referential lifecycle R and an action policy P :

$$L(r, a) = (R(r), P(a)),$$

$$R = (m, e, q, g), \quad P = (V_a, \pi), \quad (1)$$

where m is a reference mode, e is an identity bound in S_0 , q is a selector, g is an optional persistence guard, π is a fallback policy, and $V_a(e, S)$ is validity under the requested action schema. The main TRI-v3 setting uses $m \in \{\text{PRESERVE}, \text{REEVALUATE}\}$ and an explicit reject policy for an invalid preserved target. TRI-v4 adds conditional guards.

For the scalar policies evaluated here, the execution branches are

$$\text{PRESERVE}, V_a(e, S_1) \Rightarrow e, \quad (2)$$

$$\text{PRESERVE}, \neg V_a(e, S_1), \pi = \text{REJECT} \Rightarrow \perp, \quad (3)$$

$$\text{REEVALUATE} \Rightarrow q(S_1), \quad (4)$$

$$\text{CONDITIONAL}, g(e, q, S_1) \Rightarrow e, \quad (5)$$

$$\text{CONDITIONAL}, \neg g(e, q, S_1) \Rightarrow q(S_1). \quad (6)$$

An invalid preserved entity remains the discourse referent even when policy returns $\perp$. Likewise, q establishes identity under Preserve but remains executable under Reevaluate; action validity never authorizes substituting a different valid entity. Full history may contain all necessary facts while leaving this relationship computationally implicit, and correct inference still requires enforcement at the mutation boundary. TRI-v4 distinguishes $g_{\text{act}} = V_a(e, S_1)$ from $g_{\text{sel}} = [q(S_1) = e]$.

Proposition (mode-blind policy impossibility). Consider a Preserve/Reevaluate minimal pair with identical S_0 , S_1 , q , and a , where $e = q(S_0)$, $e' = q(S_1)$, and $e \neq e'$. The authorized targets are e and e' , respectively. Any deterministic post-refresh policy of the form $f(e, q, S_1, a)$, containing neither the original discourse nor a compiled mode (or

equivalent discourse-sensitive distinction), receives the same inputs for both tasks and must return the same target; it is therefore wrong on at least one member. Always-Lock and Always-Reevaluate instantiate the two complementary extremes measured in our policy controls. This establishes the need for a selective transition decision, not the necessity of our particular lifecycle serialization.

Authorization invariant (conditional guarantee). If L is correct, IDs are stable, selector/guard evaluation is correct, and the executor follows the branches above, every emitted target is referentially authorized; enforcing V_a additionally makes each executed mutation action-valid. The guarantee ends at compilation: a wrong but valid ID, identity migration, or an incomplete action schema can still pass.

Outcomes. We separately report exact target/final state, wrong-entity writes, invalid attempts, and unnecessary rejection; always rejecting avoids writes but fails every actionable task.

Method: Pre-Refresh Commitment Compilation

The controller family first compiles (u, S_0) before refresh. Minimal CTA emits binding time, selector, and a bound ID. The typed Lifecycle extension additionally emits

```
reference_mode; bound_target_id;
selector; invalidity_policy.
```

Both compilers use discourse order: “identify, refresh, act on it” preserves identity even without words such as “same,” while “refresh before deciding” reevaluates. It resolves a concrete bound ID only for preserved references.

The executor then refreshes the environment. If the compiled mode is PRESERVE, a deterministic gate checks the bound entity against action preconditions. The gate emits the bound ID when valid, returns $\perp$ under a reject policy, or invokes selector resolution only when the compiled fallback authorizes it. If the mode is REEVALUATE, the LLM actor selects from S_1 . Thus a preserved commitment is a controller constraint, not advice that a second model may reinterpret. The preserve branch also needs one model call rather than two.

TRI-v4 generalizes the record with `guard_type` and `fallback_policy`. The action-validity guard can be enforced deterministically. A selector-match guard is evaluated by the actor because the current prototype compiles natural language selectors but does not execute them as a general symbolic query language. This design choice exposes selector grounding as a measurable stage rather than using gold selector fields.

The complete controller procedure and model-facing interfaces are provided in the supplement. Gold targets and benchmark selector fields are never provided to the compiler or actor.

Information fairness. Compilers see the instruction, S_0 , action schema, and planned refresh, never gold targets or generator selector fields. Generic retains the instruction, selected ID, snapshot, selector, action, and preconditions for

its second-call actor. CTA, Lifecycle-free, and Generic each use two calls; Lifecycle-Gated skips the actor for a valid Preserve branch. Full prompts, interfaces, and executor-level oracle checks are supplementary.

Experimental Setup

Controlled task construction. Each domain specification defines an entity table, a natural-language selector, its ranking or Boolean criterion, an action, and explicit action preconditions. The generator first evaluates the selector on S_0 to obtain e_0 , applies a controlled transition, and evaluates it on S_1 to obtain e_1 . A language template determines whether the action target is e_0 , e_1 , $\perp$, or a conditionally guarded choice. Paired tasks share the domain, both states, selector, action, and schema; only discourse order and anaphoric language change. This prevents domain difficulty or target frequency from explaining the preserve/reevaluate contrast.

The five transition types serve different diagnostic roles. *Flip* changes which valid entity wins the selector. *Stable* leaves the winner unchanged and detects controllers that overreact to any refresh. *Remove* deletes the pre-refresh entity. *Invalidate* keeps its ID present while changing an action-precondition field, separating existence from validity. *Name collision* gives another entity the same display label while changing the selector winner, testing whether controllers preserve stable identity rather than surface aliases. Across anchored and dynamic language, these updates create matched cases where naive “always old” and “always new” policies have complementary failures.

Blind human construct validation. One independent volunteer naturally rewrote 50 sampled scalar instructions. Three other volunteers, uninvolved in TRI design, independently labeled 100 randomized original/rewrite items using opaque IDs without condition metadata or gold answers. They selected an entity, REJECT, or CLARIFY; confidence was optional. We report majority-gold alignment, unanimous and pairwise agreement, Fleiss’ κ , nominal Krippendorff’s α , and a consensus-only sensitivity analysis. Conditional, collection, and nested cases were not included in this human study.

Stage and write-consequence reports use a frozen mechanism-oriented error taxonomy, detailed in the supplement, that separates temporal rebinding, premature binding, invalid processing, unnecessary rejection, and selector-grounding errors.

Strong baselines and component audit. Table 1 reports the frozen v3 primary inventory. Generic validity gating, an untyped pre-refresh plan, Lifecycle-free execution, and exact historical Compile-then-act were post-primary audits; they do not replace the pre-specified Generic-versus-Lifecycle-Gated estimand. They do, however, determine how narrowly that comparison can be interpreted.

TRI-v3 language clusters. The primary test contains 160 tasks formed from 20 frozen language-template clusters, four reference styles, and eight domains per template. Within each style, flip, stable, remove, invalidate, and name-collision updates are balanced. The pre-specified statistic resamples

Controller	Qwen3.5	GLM-5.1
Generic free actor	64.4	71.9
Generic + reference mode	75.0	75.0
Generic + validity gate	65.0	73.1
Untyped pre-refresh plan	81.2	70.6
Exact compile then act	95.0	96.2
Lifecycle free actor	96.9	98.1
Lifecycle + gate	98.1	100.0

Table 1: Exact target accuracy (%) on the same 160-task v3 language set. The post-freeze Generic+reference-mode ablation adds only the mode field; Lifecycle-free is statistically tied with historical Compile-then-act, and the gate adds only 1.2–1.9 points.

whole template clusters with 10,000 bootstrap samples; task-level McNemar is secondary. Before any model call, we froze the data, runner, hashes, inference settings, and rule that prompts would not change after output inspection.

Schema transfer under shared language templates and guarded policies. An 80-task transfer set crosses the same 20 template clusters with four new domains: projects, expenses, inventory, and cloud deployments. These introduce new IDs, fields, selectors, actions, and action preconditions. TRI-v4 separately freezes 40 conditional tasks, ten template clusters, and the two guards g_{act} and g_{sel} . Every guard–update–domain combination occurs once. TRI-v4 is a secondary extension, not part of the pre-specified v3 primary comparison.

Direct-neighbor baseline. Generic Structured Ledger uses two calls. Its compiler stores the initial selected ID, complete entity snapshot, selector, requested action, and action preconditions, but omits reference mode and invalidity policy. Its actor still receives the original instruction, ledger, S_1 , and action schema. It is therefore not an information-deletion baseline: the comparison asks whether structured facts suffice without discourse commitment semantics.

The historical Compile-then-act prompt is rerun unchanged on the v3 language set (Table 1). Its cluster interval overlaps lifecycle-free execution for both models, consistent with pre-refresh compilation, rather than tuple serialization or deterministic gating alone, being the leading explanation. Because it emits binding time and a bound ID, it also serves as the simple mode-classifier-plus-ID-lock analogue for scalar cases. The typed contract contributes an auditable interface for validity and fallback; the gate is a small enforcement layer, not the principal accuracy mechanism. A matched free-form pre-refresh planner reaches only 81.2% for Qwen and 70.6% for GLM, so moving computation before refresh is not alone sufficient; nevertheless, the strong historical predecessor prevents us from claiming that the full tuple is empirically necessary for scalar tasks.

For a direct policy contrast, we also evaluate two deterministic extremes without model calls. Always-Lock + validity retains the S_0 ID and rejects it only when the refreshed action preconditions fail; Always-Reevaluate discards the binding and applies q to S_1 . These are conceptual con-

trols rather than implementations of the Binding Drift baselines. On the frozen 160-task inventory both reach 96/160 (60.0%), but their mode slices are complementary: Always-Lock reaches 80/80 anchored and 16/80 dynamic, whereas Always-Reevaluate reaches 16/80 anchored and 80/80 dynamic. The contrast is a diagnostic of selective authorization, not evidence that either unconditional policy is a competitive agent controller.

Model-facing SQLite trajectories. We freeze a 40-task subset covering all 20 templates, eight domains, both modes, and all update types. The runtime queries an in-memory SQLite database, gives returned rows to the compiler, replaces the database state during refresh, obtains a target from the gate or actor, and executes UPDATE through a mutation tool that enforces action preconditions. We inspect final database state, wrong writes, invalid attempts, and collateral modifications. Query and required refresh order are controller-orchestrated; this is a focused execution test, not a fully autonomous external benchmark.

Models, inference, and auditing. We use Qwen3.5-122B-A10B and GLM-5.1 through the same OpenAI-compatible API at temperature zero, with thinking disabled and a 1,200-token output cap. Balanced smoke tests gate each expansion. Final reports verify expected task IDs, duplicates, status, request attempts, and retries. All reported v3/v4 full files are complete and contain zero API errors and zero retries. API failures would be reported separately rather than interpreted as model behavior.

Statistical estimands. The primary estimand is Lifecycle-Gated minus Generic Structured Ledger accuracy on the Qwen language-cluster set. We resample its 20 complete template clusters rather than treating 160 templated rows as independent. The same cluster bootstrap is reported for replication and transfer sets; each interval uses 10,000 samples and a fixed recorded seed. Template-macro and task accuracy coincide in the balanced sets, but both are retained in machine-readable reports. Exact paired McNemar tests use task-level discordance and are secondary. Wilson intervals describe individual proportions. Slices by binding, explicitness, update, and domain are mechanism analyses rather than additional confirmatory hypotheses; we do not use them to select the primary claim.

Reproducible run discipline. Frozen data, protocols, runners, and smoke subsets have recorded SHA-256 hashes. Reports retain raw outputs, attempts, retries, stage errors, and SQLite traces; interrupted rows are auditable by ID. The anonymous artifact maps every main-table value to a raw run and report script, includes the human-analysis code and de-identified responses, and excludes API keys and private coordination files. Post-primary audits are labeled as such.

Results

Primary and Transfer Results

Structured facts are not a commitment. Table 2 shows the same mechanism across models. On the primary language-cluster test, Generic Structured Ledger reaches

Benchmark	Model	Generic	CTA	Lifecycle/Guarded	Δ	Cluster 95% CI
Language clusters (primary)	Qwen3.5	64.4	95.0	98.1	+33.8	[+18.1, +50.0]
Language clusters	GLM-5.1	71.9	96.2	100.0	+28.1	[+18.1, +38.1]
Core replication (new states)	Qwen3.5	47.5	70.8	71.2	+23.8	[+16.2, +31.2]
Core replication (new states)	GLM-5.1	70.0	94.2	97.1	+27.1	[+22.1, +32.1]
Schema transfer (shared templates)	Qwen3.5	46.2	–	82.5	+36.2	[+25.0, +47.5]
Model-facing SQLite	Qwen3.5	67.5	–	100.0	+32.5	[+15.0, +50.0]
Model-facing SQLite	GLM-5.1	65.0	–	100.0	+35.0	[+17.5, +52.5]
Guarded conditional policies	Qwen3.5	52.5	–	85.0	+32.5	[+15.0, +52.5]

Table 2: Exact target or final-state accuracy (%). CTA is shown where it was run on the identical scalar language inventory; Δ remains Lifecycle/Guarded minus Generic. Confidence intervals resample complete language-template clusters for v3 and state-instance clusters for the new core replication.

64.4% for Qwen and 71.9% for GLM, despite carrying identity, facts, selector, preconditions, and the original instruction. Lifecycle-Gated reaches 98.1% and 100.0%. Paired discordance is 3 generic-only versus 57 lifecycle-only for Qwen ($p = 6.25 \times 10^{-14}$), and 0 versus 45 for GLM ($p = 5.68 \times 10^{-14}$). Qwen improves from 33.8% to 100.0% on anchored references while remaining comparable on dynamic references (95.0% versus 96.2%). GLM improves from 56.2% to 100.0% anchored and from 87.5% to 100.0% dynamic. The gain is therefore not explained by always preserving old IDs.

Because the Lifecycle compiler receives explicit reference-mode and invalidity semantics, this comparison does not by itself identify the causal contribution of serialization, prompt supervision, and the deterministic gate separately. The historical compile-then-act results in Table 1 provide a strong neighboring comparison and motivate the narrower interpretation: explicit pre-refresh compilation is the main performance mechanism, while lifecycle state makes its authorization and validity decisions auditable and enforceable.

The post-primary component audit sharpens this conclusion. A validity-only gate changes Generic accuracy by only +0.6 points for Qwen and +1.2 for GLM. Lifecycle state with a free actor reaches 96.9% and 98.1%; deterministic enforcement adds only +1.2 and +1.9 points. The new Generic+reference-mode ablation reaches 75.0% for both models, with paired cluster differences of +10.6 [+5.0, +16.9] and +3.1 [−2.5, +9.4] points. Thus explicit mode classification is useful but model-dependent and insufficient to explain CTA’s full gain. The call-matched untyped plan reaches 81.2% and 70.6%, 15.6 and 27.5 points below Lifecycle-free with cluster intervals excluding zero, showing that pre-refresh timing alone is insufficient. Exact historical CTA reaches 95.0% and 96.2%, tied with Lifecycle-free; the audit therefore supports commitment compilation as the main mechanism, with typed lifecycle state adding auditable validity and fallback rather than explaining the full accuracy gap.

The deterministic policy controls further separate TRI from unconditional persistence. Always-Lock + validity and Always-Reevaluate each reach only 60.0% overall. Their equal totals hide opposite errors: lock is 100.0% on anchored and 20.0% on dynamic references, while reevaluation is 20.0% anchored and 100.0% dynamic. TRI’s empiri-

cal target is the discourse- conditioned choice between these transitions, not locking by default.

Because the aggregate includes a normative invalid-target policy, we also separate its 128 actionable core items: Generic/Historical-CTA/Gated reach 74.2/98.4/97.7% for Qwen and 72.7/99.2/100.0% for GLM. The complete 32-item policy slice is supplementary; this post-primary sensitivity does not replace pre-specified total accuracy.

The strongest generic failures occur when the bound entity remains valid but no longer wins the selector: GLM obtains 18.8% on anchored flips and 25.0% on name collisions, while the lifecycle gate is perfect. Implicit commitments are also harder for generic state (42.5% anchored for GLM) than explicit ones (70.0%). A field-corrected stage audit is important here: Generic stores the initial target as `selected_entity_id`, not the lifecycle-only `bound_target_id`. Conditioning on the former being correct, Qwen drifts to the refreshed leader on 15/16 anchored flips and 14/16 name collisions; GLM does so on 3/16 and 7/16, respectively, while stable controls show 0 drift. These patterns match the predicted distinction between storing evidence and enforcing what that evidence means for denotation.

A larger frozen replication confirms conditional drift.

After selecting the mechanism, we froze 240 core-only tasks over ten unseen schemas and four independently parameterized states per schema. All old targets remain present and action-valid; the 40 state instances are the resampling units. Generic binds 107/120 Qwen and 120/120 GLM anchored targets correctly. Within the resulting flip/name-collision opportunities it then drifts to the refreshed winner 43/72 times (59.7%, state-cluster 95% CI [46.5, 72.4]) and 38/80 (47.5%, [35.0, 61.3]); stable final-target errors are 4/40 and 2/40. Exact CTA and Gated have zero such drift after a correct binding. Overall CTA improves over Generic by 23.3 points [16.2, 30.4] and 24.2 [19.6, 28.7]. Qwen’s lower dynamic accuracy exposes compound-selector grounding rather than temporal drift, so this post-primary replication strengthens the existence claim without converting the synthetic benchmark into a prevalence estimate.

Human judgments support the core transition, with a fallback boundary. Across 100 blinded items, Fleiss’ $\kappa = .708$, Krippendorff’s $\alpha = .709$, and majority-gold agree-

ment is 86% (91.5% among 94 determinate items); original and rewritten language obtain 84% and 88%. On 50 frozen volunteer rewrites, Generic/CTA/Lifecycle-free/Gated reach 60/90/88/90% for Qwen and 74/98/92/94% for GLM. Direct CTA-minus-Gated intervals are 0.0 $[-11.1, +11.1]$ and +4.0 $[-4.3, +14.8]$ points; CTA-minus-Lifecycle-free intervals also span zero. Against 48 rewrite majorities, CTA reaches 89.6/93.8% versus Gated’s 85.4/85.4%; on 41 majority-actionable items, the values are 92.7/92.7% versus 82.9/82.9%. Lifecycle errors mostly miscompile refresh-first rewrites as PRESERVE; CTA errors concentrate in invalid-target policy. Action-invalid REJECT obtains only 55% majority-gold and 25% unanimity, so we treat Preserve/Reevaluate as the human-supported core and Reject as a normative action policy.

External and Compositional Boundaries

The external and compositional evaluations do not show a stable extension of the scalar result. In the 24-task ToolSandbox-style pilot, Generic, Lifecycle, Gated, and Untyped obtain 79.2/87.5/91.7/91.7% for GLM and 91.7/83.3/91.7/79.2% for Qwen, respectively. Gated permits one GLM and two Qwen wrong writes. We therefore treat this pilot as an external validity boundary check: the TRI diagnosis can be instantiated in a stateful tool loop, but the controller advantage is not stable once tool choice and up-stream compilation are exposed.

We additionally froze a 96-task, single-user-turn ToolSandbox-based 2×2 extension with an auditable binding opportunity. Qwen and GLM full-history agents produced zero TRI mechanism errors in 70 and 73 eligible opportunities, respectively; matched Qwen and GLM Generic Ledgers produced zero in 64 and 87. The external runs did produce real wrong-entity writes (6, 13, 5, and 4), but each was an initial selector or tool-order error outside the TRI denominator. We report this negative external result as a boundary, not as prevalence evidence: the controlled Generic result is model- and controller-dependent and does not imply that every full-history LLM will exhibit TRI.

We also audited unmodified public inventories before adaptation. At pinned ToolSandbox commit 165848b, none of 129 semantic scenario families contains a strict post-binding external transition plus a later substitutable mutation; one task naturally preserves “them” after the selected contacts change category. Across AppWorld’s 244 generator families, one public Todoist family is a stronger natural near-match: tasks selected as assigned-to-me are reassigned and then “leave a comment there” must retain the same task IDs even though they no longer satisfy the initial selector. In 42 released trajectories from 14 experiment configurations, 16 correct target-binding operations are followed by 16 same-ID comments and zero post-binding substitutions; most failures omit targets before binding. Thus an independent benchmark supports the task structure, but neither inventory estimates strict TRI prevalence. At current τ^3 -bench commit cf71a80, a separate audit covers 2,449 airline/retail/telecom tasks and 10,832 released GPT-4.1/Claude/o4-mini trajectories. It finds zero strict same-role TRI opportunities; eight telecom tasks naturally bind a bill,

let the user pay, and continue on an associated line, but bill and line are different roles with no competing selector target.

Finally, a custom AppWorld case study crosses reference mode, Stable/Flip, and two paraphrases in two application/selector clusters: Todoist earliest-due tasks (frozen first) and a post-primary Simple Note alphabetical-title extension. Across 32 Qwen/GLM full-history trajectories using native APIs and database IDs, 31 write the authorized target. Of 24 trajectories with a correct, correctly timed observable binding, conditional TRI is 0/24 (0/4 in Preserve/Flip). The sole wrong write follows synchronization before binding and is therefore a delayed-binding/tool-order error, not post-binding drift. This is a custom adaptation, not an AppWorld leaderboard score, but it confirms that the external null is not an artifact of label-only scoring.

A lower-intervention addendum then removes the binding sidecar and all prompt language about TRI, temporal authorization, or reference modes. Ordinary selector APIs expose the returned ID without a measurement call. Across the same two apps and models, 28/32 trajectories form a correctly timed binding and conditional TRI is 0/28 (0/6 Preserve/Flip). Qwen makes two real Flip wrong writes, but in each matched app it synchronizes before its first selector call; the corresponding Stable row has the same order defect but masks it because the winner is unchanged. Thus these are pre-binding temporal-order errors, not post-binding drift.

The frozen Qwen TRI-v5 stress test contains two refreshes, a monitoring referent, an unrelated tool call, and a final mutation. Generic obtains 32/40 final states, one wrong write, and four unnecessary rejections; scalar Lifecycle obtains 28/40, six wrong writes, six unnecessary rejections, and one parse/internal error. The difference is -10.0 points with a template-cluster interval of $[-35.0, 12.5]$. A role-indexed held-out addendum improves Qwen from 35/40 to 39/40 final states, but recovered GLM is tied at 40/40; this motivates role indexing without establishing stable superiority over simpler compilation. These results bound the current claim to controller-orchestrated, single-refresh, scalar mutations rather than general multi-referent reliability.

Error and Safety Analysis

Stage analysis localizes the remaining error. On language clusters, GLM compiles mode, ID, and policy correctly on all tasks and has no actor failure. Qwen also compiles all lifecycle fields correctly; its three final errors occur in the dynamic actor. Under schema transfer with shared templates, mode remains 100%, but bound-ID accuracy falls to 55.0% for explicit and 50.0% for implicit anchored references. The errors concentrate in compound selectors such as “lowest-stock reorderable item” and “oldest failing deployment”: Qwen sometimes chooses the lowest item before filtering *reorderable*, or the oldest deployment before filtering *failing*. The gate then rejects the ineligible ID. This produces conservative errors rather than wrong writes, but it shows that TRI semantics and selector grounding are separate research problems.

Transfer and real write consequences. On the four new schemas under shared templates, lifecycle control improves

Qwen from 46.2% to 82.5%; the cluster interval excludes zero. In model-facing SQLite runs, Generic Structured Ledger gives Qwen 27/40 final states with 13 wrong-entity writes, while GLM gives 26/40 with 8 wrong-entity writes. A conditional trace audit separates the referential core from invalidity policy. Both models compile the correct initial ID on all 20 anchored tasks. Among the eight flip/name-collision cases in which that entity remains present and action-valid, Generic writes the refreshed selector winner in 8/8 Qwen and 6/8 GLM trajectories; stable controls have 0/4 wrong writes for each model. The remaining 5 and 2 wrong writes occur after removal or invalidation and therefore test the benchmark’s reject policy rather than the core post-binding transition. Lifecycle- Gated reaches 40/40 for both models with no wrong write, invalid attempt, unnecessary rejection, or collateral modification. The paired exact test for Qwen gives $p = 2.44 \times 10^{-4}$; the GLM paired cluster difference is reported in Table 2. These are actual post-refresh database mutations, not target-string labels. Generic plus a validity-only gate still produces 13 Qwen wrong writes because replacement targets are often action-valid; both Lifecycle variants obtain 40/40. This demonstrates a correctly compiled contract, not general reliability against compiler or tool failures.

Discussion

Commitments benefit from explicit controller state. Generic already carries enough evidence in principle; its failures instead show that an implicit relationship remains behaviorally unstable: is a selector still an instruction to search, or provenance for a committed ID? Pre-refresh compilation answers this once. Typed lifecycle state makes the answer auditable and separates revisable beliefs from discourse commitments without claiming that the tuple is the only sufficient representation.

Gates move, rather than erase, the reliability bottleneck. A gate cannot repair a wrong mode, ID, or guard; an incorrectly bound yet valid ID can pass. It only prevents a later actor from violating a correctly compiled contract and enforces action validity. Under schema transfer this sometimes converts grounding errors into rejection, shifting rather than eliminating risk and motivating clarification and executable selectors. In the new replication’s SQLite replay, CTA/Gated produce no core TRI write after a correct binding, yet Qwen still makes 8/17 and GLM 14/7 total wrong writes, mainly from dynamic over-locking or selector errors. Enforcement is therefore conditional, not a general write-safety guarantee.

Boundary results constrain the design claim. The ToolSandbox and multi-referent results belong to the empirical finding, not merely to future work: a correctly designed gate cannot compensate for an incorrectly compiled contract or an upstream tool error, and a scalar contract does not compose automatically across referent roles. Within the supported scalar setting, gating reduces primary requests from 320 to 240 per model.

Limitations

Scope and construct validity. TRI is a controlled diagnostic, not a prevalence estimate. It still simplifies ambiguity, repair, partial observability, and identity migration. Human validation covers 100 scalar original/rewrite items, not conditional, collection, or nested semantics; low agreement on invalid-target rejection limits fallback claims. Primary experiments use one target and one refresh, while a negative multi-referent stress test does not establish composition.

Causal attribution and external validity. The primary 20 template clusters and post-primary 40-state replication use temperature-zero calls and do not characterize execution variance or all model families. The replication was frozen after the mechanism hypothesis was selected and remains synthetic. We reproduced the 25 deterministic offline assertions in the Binding Drift artifact after a filename adaptation, but have not rerun its Bedrock model sweep. Its deterministic re-verification reads a gold target and is therefore an oracle upper bound, while Entity Lock corresponds to our Always-Lock extreme. Compound-selector grounding remains a bottleneck. Fixed query-refresh order, local SQLite, ToolSandbox-style tasks, and a two-cluster custom AppWorld case study do not establish real-traffic prevalence. The public trace near-matches have no Stable/Flip contrast, the official τ^3 inventory has no strict opportunity, and the external adaptations have null conditional TRI results; opportunity coverage and controller state representation materially affect whether the phenomenon is observable.

Conclusion

World updates need not rebind targets. In the controlled benchmark, minimal pre-refresh CTA has the highest or tied scalar point estimate, without a significant advantage over typed variants. Typed lifecycle state exposes richer policy but adds compilation errors; gates enforce only correct contracts. Composition and external validity remain open. The supported design principle is narrow: decide whether a description has become an identity commitment before incorporating later observations. The corrected Generic audit and 240-task new-state replication confirm post-binding drift when the initial ID is correct, while the external ToolSandbox-style and AppWorld-backed studies find no such drift in the tested full-history agents. Released AppWorld traces exhibit the underlying persistence structure, while the τ^3 audit shows that even a large dual-control suite need not contain a strict same-role TRI opportunity. The result is therefore a controller- and model-conditional mechanism diagnosis, not a universal LLM failure or an autonomous-agent safety guarantee.

References

- Babu, R. S.; and Indukuri, S. 2026. Entity Binding Failures in Tool-Augmented Agents. *arXiv:2606.30531*.
- Barres, V.; Dong, H.; Ray, S.; Si, X.; and Narasimhan, K. 2025. τ^2 -Bench: Evaluating Conversational Agents in a Dual-Control Environment. *arXiv:2506.07982*.
- Beurer-Kellner, L.; Fischer, M.; and Vechev, M. 2023. Prompting Is Programming: A Query Language for Large Language Models. In *Proceedings of the 44th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 1946–1965.
- Dasigi, P.; Liu, N. F.; Marasović, A.; Smith, N. A.; and Gardner, M. 2019. Quoref: A Reading Comprehension Dataset with Questions Requiring Coreferential Reasoning. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 5925–5932.
- Gardner, M.; Artzi, Y.; Basmova, V.; Berant, J.; Bogin, B.; Chen, S.; Dasigi, P.; Dua, D.; et al. 2020. Evaluating Models’ Local Decision Boundaries via Contrast Sets. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 1307–1323.
- Grosz, B. J.; Joshi, A. K.; and Weinstein, S. 1995. Centering: A Framework for Modeling the Local Coherence of Discourse. *Computational Linguistics*, 21(2): 203–225.
- Indukuri, S. 2026. Binding Drift in Multi-Step Tool-Augmented Agents. *GitHub repository*.
- Joshi, M.; Chen, D.; Liu, Y.; Weld, D. S.; Zettlemoyer, L.; and Levy, O. 2020. SpanBERT: Improving Pre-training by Representing and Predicting Spans. *Transactions of the Association for Computational Linguistics*, 8: 64–77.
- Kamp, H.; and Reyle, U. 1993. *From Discourse to Logic: Introduction to Modeltheoretic Semantics of Natural Language, Formal Logic and Discourse Representation Theory*. Kluwer Academic Publishers.
- Khattab, O.; Singhvi, A.; Maheshwari, P.; Zhang, Z.; Santhanam, K.; Vardhamanan, S.; Haq, S.; Sharma, A.; et al. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *International Conference on Learning Representations*.
- Lee, K.; He, L.; Lewis, M.; and Zettlemoyer, L. 2017. End-to-end Neural Coreference Resolution. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 188–197.
- Li, M.; Zhao, Y.; Yu, B.; Song, F.; Li, H.; Yu, H.; Li, Z.; Huang, F.; and Li, Y. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 3102–3116.
- Liu, X.; Yu, H.; Zhang, H.; Xu, Y.; Lei, X.; Lai, H.; Gu, Y.; Ding, H.; Men, K.; et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*.
- Lu, J.; Holleis, T.; Zhang, Y.; Aumayer, B.; Nan, F.; Bai, H.; Ma, S.; Ma, S.; Li, M.; Yin, G.; Wang, Z.; and Pang, R. 2025. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, 1160–1183.
- Packer, C.; Wooders, S.; Lin, K.; Fang, V.; Patil, S. G.; Stoica, I.; and Gonzalez, J. E. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*.
- Patil, S. G.; Zhang, T.; Wang, X.; and Gonzalez, J. E. 2023. Gorilla: Large Language Model Connected with Massive APIs. *arXiv:2305.15334*.
- Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; et al. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*.
- Ribeiro, M. T.; Wu, T.; Guestrin, C.; and Singh, S. 2020. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 4902–4912.
- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*.
- Sierra Research. 2026. τ^3 -Bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *GitHub repository*.
- Sohail, S.; and Haider, G. 2026. Bounded Autonomy for Enterprise AI: Typed Action Contracts and Consumer-Side Execution. *arXiv:2604.14723*.
- Sumers, T. R.; Yao, S.; Narasimhan, K.; and Griffiths, T. L. 2024. Cognitive Architectures for Language Agents. *Transactions on Machine Learning Research*.
- Tang, Z.; Zhao, Q.; Franco, G.; Wijaya, D.; Mueller, A.; Schuster, S.; and Kim, N. 2026. Do Language Models Track Entities Across State Changes? In *International Conference on Machine Learning*. *ArXiv:2605.30233*.
- Trivedi, H.; Khot, T.; Hartmann, M.; Manku, R.; Li, V. D.; Gupta, S.; Sabharwal, A.; and Balasubramanian, N. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. *arXiv:2407.18901*.
- Uddin, M. N.; Saeidi, A.; Blanco, E.; and Baral, C. 2026. Ledger-Agent: Structured State for Policy-Adherent Tool-Calling Agents. *arXiv:2606.20529*.
- Willard, B. T.; and Louf, R. 2023. Efficient Guided Generation for Large Language Models. *arXiv:2307.09702*.
- Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2024. Tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv:2406.12045*.
- Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafraan, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.
- Zhou, S.; Xu, F. F.; Zhu, H.; Zhou, X.; Lo, R.; Sridhar, A.; Cheng, X.; Ou, T.; Bisk, Y.; et al. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*.